Data-Structures & Algorithms

with

Manoj Prabhakaran

Lecture 25

Hash Tables (ctd.)

Hash Tables

- Recall Hash Table template:
- Initialise an array A of size m, and a hash function H drawn from a family $\mathscr{H}$
- To search for the location of a key, compute an index H(key) $\in$ {0,...,m-1}
  - H(key) should appear to be uniformly random, independent of the other keys
  - Load factor n/m determines the expected number of collisions at each index. If it is too large, rehash: increase m, pick H from a corresponding $\mathscr{H}$, and reinsert all elements into a new table of size m
  - Need to handle collisions
- Today:
  - Rehashing, and handling collisions

Two-Stage Hash Functions

- Recall that to limit collisions, even using an ideal hash function, the load factor n/m should be upper bounded by a small constant (say 1)
- So, as n grows, m will have to grow too
- When m changes, the hash function will have to change too!
- This motivates a *two-stage hash function:* $H(key) = g_m(h(key))$, where $h: K \to R$ for a fixed range R (usually 64-bit integers) and $g_m: R \to \{0,..,m-1\}$
  - h is drawn from a good hash function family that doesn't depend on m
  - g_m is a (fixed, fast) *regular* function: i.e., sends a uniformly random input to a uniformly random output. Compresses $r \in R$ to an index.
- In some cases (e.g., C++ STL defaults) h is not good (i.e., not approximating uniform), but only collision resistant. Then g_m should ideally be drawn from a good hash function family (not so in STL!)

Two-Stage Hash Functions

- In std::unordered_set and std::unordered_map, while instantiating the template, one can/must provide a hash function h
  - Defaults are available for basic types, pointers, std::string etc.
  - But not for vectors or structs/classes
- The function g_m is fixed by the library for each m (e.g., modulo m)
  - In some implementations (with weak default h), m is chosen to be a prime number, to prevent g_m from creating collisions on regularly spaced inputs
- When m changes, the hash function $H = g_m \circ h$ needs to be redefined, and recomputed for every item in the hash table
  - h(key) will be *cached* along with the item, so that rehashing can avoid recomputing h, and only compute g_m

Attendance Break!

Login using CSE LDAP id at

https://www.cse.iitb.ac.in/course_vm/cs213

Read the question, choose the answer and submit.
The points are mainly for attempting,
with a small bonus for getting it right.

The system will select a few random students
who submitted

The selected students should come to the front

If you are not in the classroom, a report will be sent to DADAC!



Handling Collisions

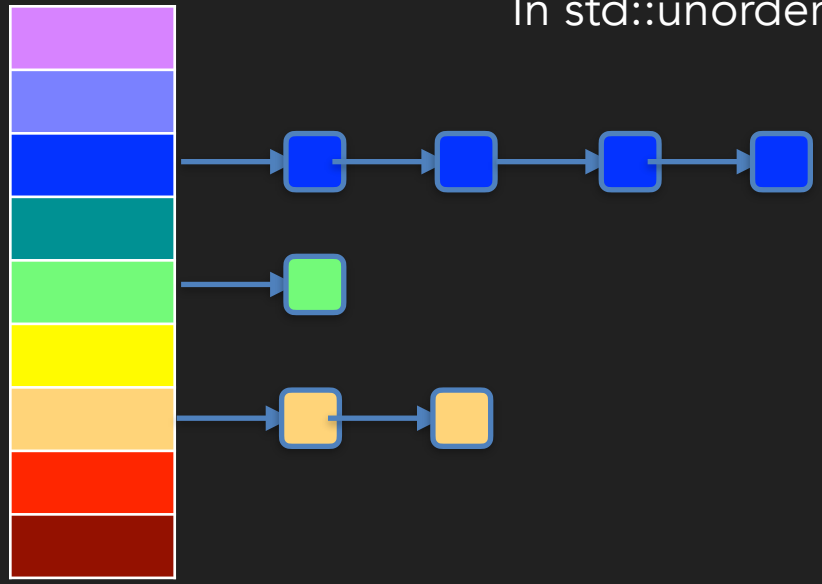
- What about the few collisions that do occur, even when we are lucky?
  - Simplest paradigm: At each index, allow multiple items to be held
  - Alternate paradigm: When collision, find another spot (next lecture)
- In std::unordered_map, items falling into the same index are held in a linked-list
  - Other containers would work too
    - E.g., in Java, switch to balanced BST when large number of items in an index (possibly because adversarial inputs broke uniform hashing assumption)
  - If the number of items in this container is O(1), looking through them will cost only O(1) time, and so total look up time is still O(1)
    - Assuming uniform hashing and bounded load factor, indeed O(1) expected list size, and further, with overwhelming probability, only at a few indices the number of items could be slightly larger: in $O(n^{\delta})$ indices O(log n/log log n) items

Handling Collisions

- A secondary goal in a Hash Table: O(n) time enumeration of all the items stored in the table (in arbitrary order)
  - So each bucket should be enumerable
  - Should be able to move to next bucket from the end of one bucket
- Idea: A single global list of all items, formed by lists from all the (non-empty) indices linked together
  - At each non-empty index, head and tail of its segment in the global list can be stored
  - When an index is populated for the first time, its list can be inserted at the head/tail of the global list

Chaining

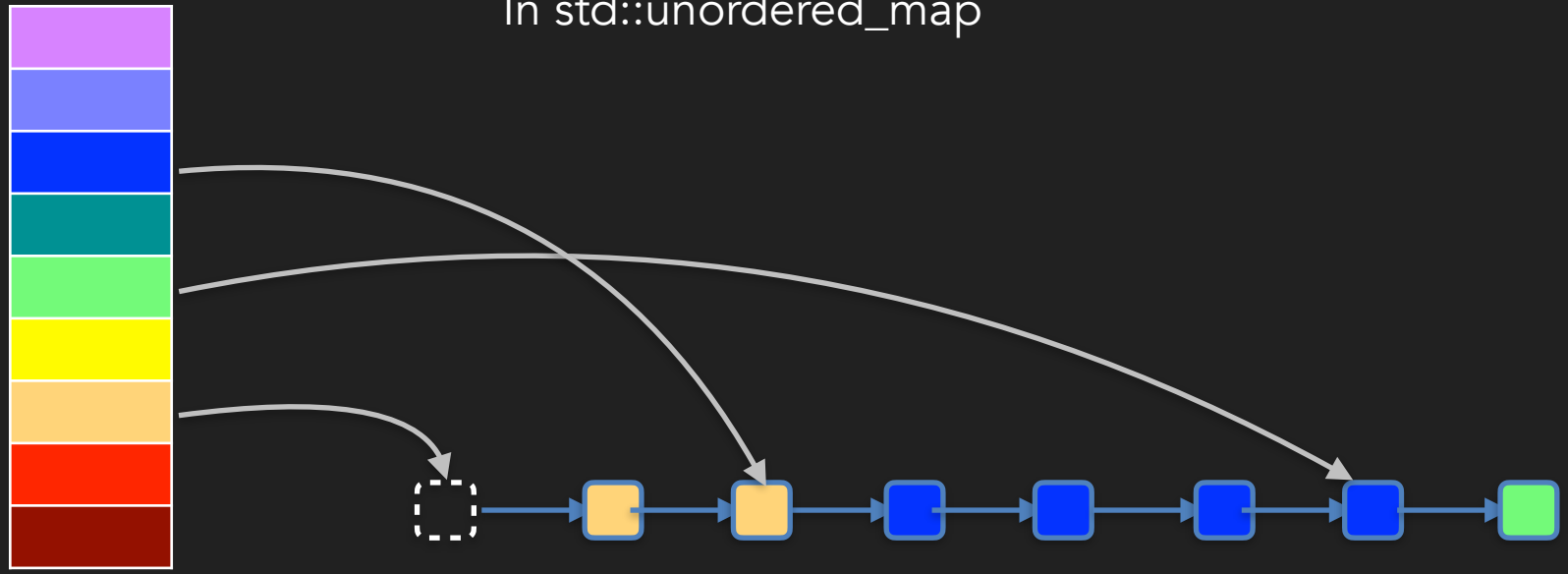
In std::unordered_map



Each index holds zero or more items, in a chain

Chaining

In std::unordered_map

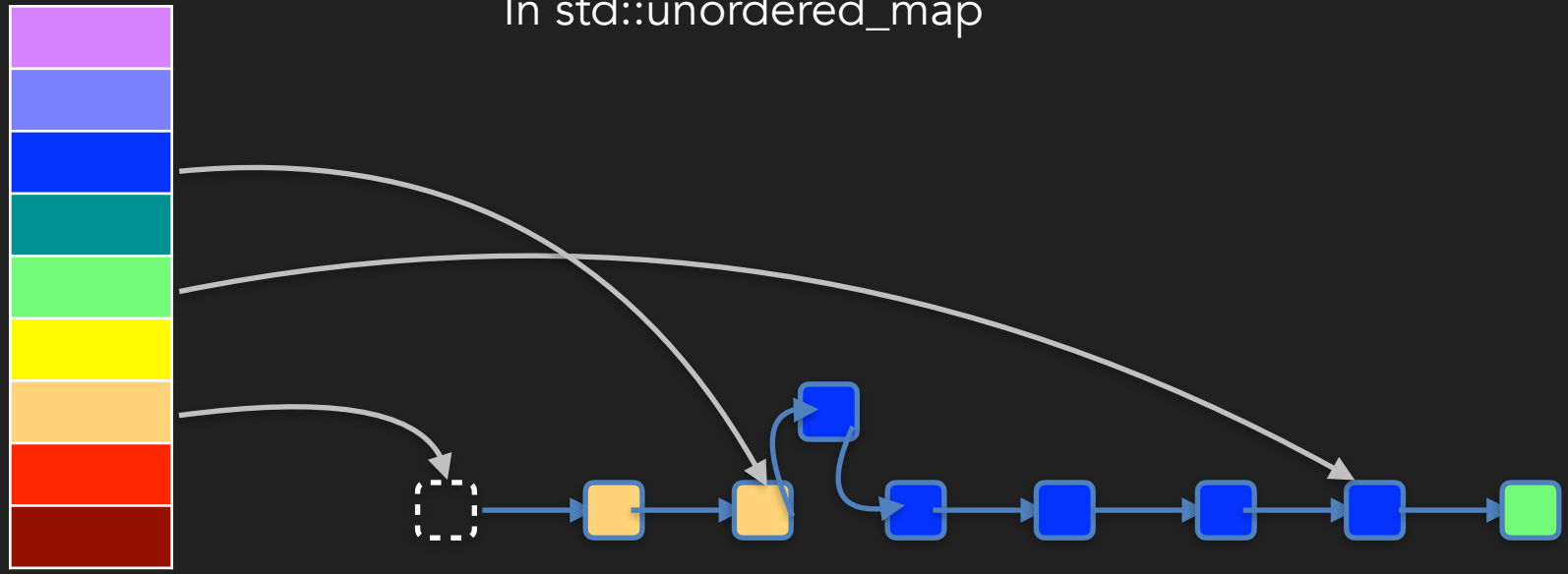


All items held in a single global list to enable O(n) time enumeration

While searching in a single sub-list, should be able to tell where it ends

Chaining

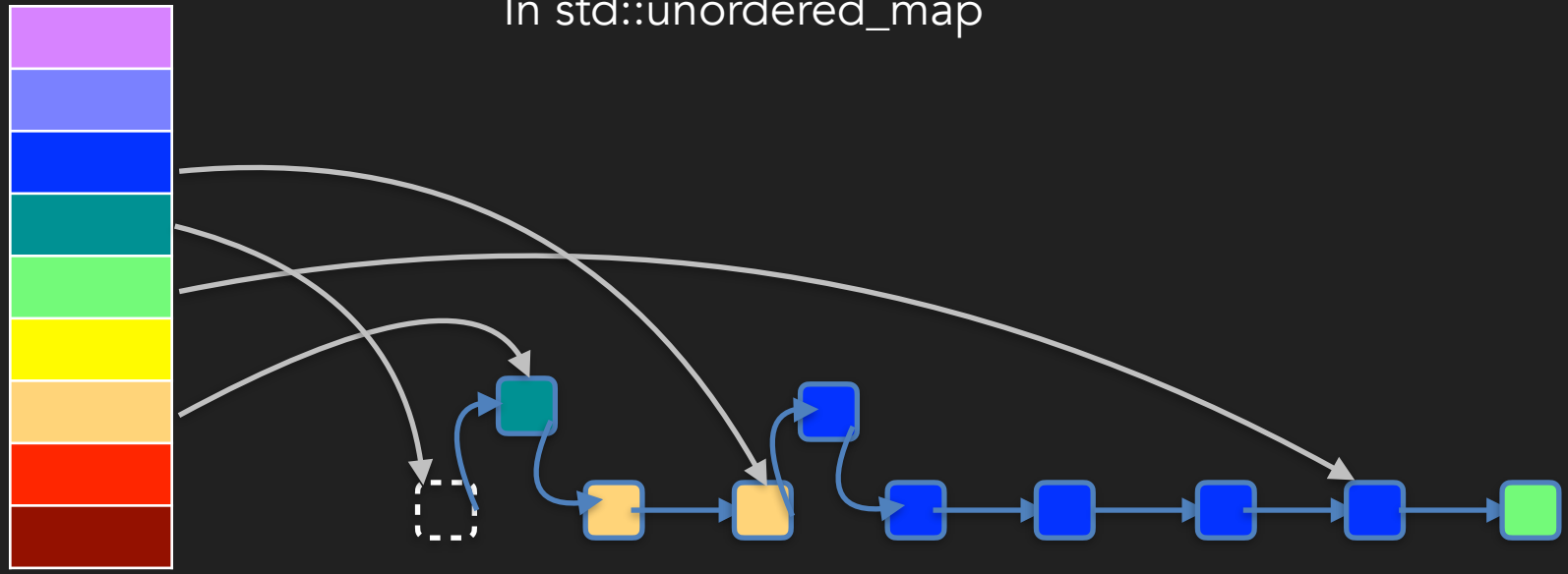
In std::unordered_map



New items at each index are added to the head of their sub-lists

Chaining

In std::unordered_map



New sub-lists are created at the head of the list

Optimisations

- In each item, store cached_tag = h(key), where $H = g_m \circ h$
- While searching in a list:
  - Compare cached_tags (integers) before comparing keys (which may be more complex)
  - Stop when g_m(cached_tag) changes -- it is the next bucket!
    - In each index store only a pointer to the start of its segment in the global list (or a nullptr): no need for storing the tail.
- When rehashing
  - Copy all items from the current global list into a new hash table, but while recomputing H, only need to compute g_m(cached_tag) now